GitHub Copilot ACP

Your Tool Just Got an AI Agent

One flag. Any editor. Full agent capabilities.

45 min • Developers • Platform Engineers • Tool Builders

🤔 The Core Question

"I build tools outside VS Code — how do I give them Copilot's full agent capabilities?"

Every role needs something different: real integration speed, enterprise trust, or multi-agent coordination.

ACP gives each of you exactly what you need.

Zed Developer
Connect without months of custom work

Platform Engineer
Ship safely with policy-enforced permissions

Orchestrator
Coordinate agents across repositories

2 weeks
vs 3 months to ship before ACP

14 lines
of TypeScript to connect

3 lines
of policy to secure automation

📖 Table of Contents

Click any section to jump directly there

PART 1

Zed Developer

From locked out to full agent experience in 14 lines of code

🚀

One Command

copilot --acp --stdio

⚡

Streaming Responses

Real-time agent output

📦

Any Editor

Standard protocol for all

```
Connect any editor to Copilot
↳ 2 weeks instead of 3 months
```

From Command to Live Agent Workflow

acp-protocol

```
→ {method: initialize, params: { ... }}
← {result: initialized}

→ {method: prompt, text: Refactor fetch}
← {event: thinking, text: Analyzing ... }
← {event: tool, tool: read_files, path: *.ts}
← {event: thinking, text: Found 3 fetches}
← {event: text, delta: function getUsers() {
  return Promise.all([
    fetch(/api/users),
    fetch(/api/posts)
  ])
}}
```

Streaming protocol

Real JSON-RPC messages: → client, ← server

Visible agent internals

See thinking, tool calls, and code as it streams

Language-agnostic

Any editor can implement this protocol

From Locked-In to Free

Before ACP

- Stuck waiting for IDE team
- 3+ months
  Build custom agent API
- Limited to existing features

After ACP

- Connect in 2 weeks
  Standard protocol, any tool
- Full agent reasoning available
- Your control
  No waiting for platform updates

90%	2-3 weeks	∞
less integration code	time to market	use cases

PART 2

Platform Engineer

Three permission strategies prove ACP is production-safe

Interactive Approval

Human reviews each request

Policy-Based

Auto-approve safe ops

Tiered by Env

Different rules per context

```
Permission callback gates every action
↳ 3 lines to auto-approve reads, prompt writes
```

Permissions Done Right

❌ The Challenge

Ship agents to production safely

Balance:
Power vs. policy enforcement

💡 The Approach

Permission callback intercepts requests

Three strategies:
Interactive, Policy-based, Tiered by env

✅ The Result

Auditable per-session permissions

Your rules.
Enforced at the agent boundary

Permission Policies That Scale

policy.ts

```
const SAFE = [read, search];
const BLOCKED = [delete];

async requestPermission(p) {
  if (SAFE.includes(p.tool))
    return approved;
  if (BLOCKED.includes(p.tool))
    return cancelled;
  return await promptUser(p);
}
```

💾 Auto-approve safe ops

Reads never prompt

🛑 Auto-block dangerous ops

Deletes always blocked

👤 Human decides the rest

Writes prompt the user

PART 3

Multi-Agent Orchestrator

One protocol, many agents, coordinated work across repos

Broadcast

Prompt all workers at once

Synthesize

Orchestrator merges results

Compose

Standard sessions, no special mode

```
Spawn one agent per repo
↳ Coordinate results naturally
```

Spawning Multi-Agent Sessions

orchestrator.ts

```
const workers = repos.map(repo ⇒ ({
  process: spawn(copilot, [--acp, --stdio]),
  repo
}));

for (const worker of workers) {
  const sess = await connection.newSession({
    cwd: worker.repo.path,
    mcpServers: [filesystemServer(repo.path)]
  });
}
```

📦 One process per repo

Isolated context and state

🔀 Same protocol everywhere

No special multi-agent mode

🧩 Compose at will

Standard ACP sessions compose naturally

Five Scenarios, One Operator Playbook

SCENARIO.md

```
1. service-audit-and-api-alignment
2. payment-expansion
3. loyalty-feature-rollout
4. inventory-sync-coordination
5. security-audit-shared-upgrade

Suggested: 1 → 2 → 3 → 4 → 5
```

🏁 Start with architecture truth

Scenario 1 creates a shared baseline across all repos

🔁 Escalate complexity

Scenarios 2-4 layer feature rollout and dependency routing

🚨 End with critical path ops

Scenario 5 proves security sequencing under pressure

Scenario 1: Broadcast First, Synthesize Second

The fastest way to align eight shopfleet services

```
$ Broadcast to all workers
> "Audit this microservice: purpose, APIs, dependencies, risks"
🤔 Workers streaming:
  users: profile + auth ownership
  orders: fulfillment + payment handoff
  storefront: UI + checkout orchestration
---
$ Set Orchestrator Focus
> "Synthesize dependency map + alignment gaps + next 3 actions"
✅ One architecture narrative from seven isolated contexts
```

Operator pattern: shared prompt -> worker evidence -> orchestrator synthesis

Scenario 3: Loyalty Rollout with Cross-Repo Cascades

acp-manifest.json

```
{
  "name": "shopfleet-storefront",
  "dependsOn": ["shopfleet-users", "shopfleet-products"]
}

// Broadcast: implement loyalty points
// Route plan: storefront ← users, products, payments
// Cascade: service-specific prompts generated per dependenc
```

🧠 Graph-aware prompts

Routes are generated from repo relationships, not guesswork

🎯 Repo-specific execution

Each worker gets the same intent but tailored implementation context

📏 Consistency checks

Orchestrator catches schema drift across LoyaltyAccount and PointsTransaction

Scenario 5: Shared Library Security Upgrade

Order of operations is the difference between safety and outage

❌ Ad-hoc rollout

1. **Patch services in parallel**
   Inconsistent hash behavior
2. **Discover breakages after deploy**
   Login failures surface late
3. **No migration plan**
   Users forced into resets
4. **Manual coordination**
   High incident risk

⚠️ Uncontrolled security blast radius

✅ Orchestrated rollout

1. **Patch shopfleet-shared first**
   Single source of crypto truth
2. **Broadcast diagnostic audit**
   Find all hash usage before rollout
3. **Dual-verify transition**
   Old SHA + new bcrypt during migration
4. **Orchestrator validates all services**
   Evidence before production

✅ Controlled migration with audit trail

Critical path: shared library -> dependent services -> validation broadcast

Live Screenshot Storyboard for Your Polyrepo Demo

1️⃣

Topology

- Full app layout: orchestrator + 3+ workers
- Dependency graph with node and edge counts
- Banner showing unloaded dependency neighbors

2️⃣

Coordination

- Routing plan panel with edited downstream prompts
- Broadcast results with coalesced worker outputs
- Orchestrator card in Synthesizing state

3️⃣

Agent States

- Worker card with dependsOn and dependedBy pills
- Unloaded dep chip with Load as Worker action
- Session panel showing Restore vs Re-spawn

🧭 If you want backup visuals, cli-acp has walkthrough captures under docs/images/walkthrough/01-06 while you replace with live local shots.

PART 4

Ship Safely

Decision table and alternatives: When to use ACP, when not to

Use ACP

Non-VS Code, multi-agent, CI/CD

Maybe Not

Already in VS Code is simpler

Choose Wisely

Know your alternative

```
Four personas, one next step each
↳ Flywheel: more clients → more agents
```

Choose Your Path

🎯

Use ACP Now

Non-VS Code tool, multi-agent coordination, Claude API, enterprise policy

🤔

Consider Alternatives

Already VS Code, IDE-specific needs, single-agent only, built-in copilot works

📚

Start Small

Read spec, try SDK for your language, deploy to dev/test, add policies before prod

✅ What You Can Do Today

Today

- Follow agentclientprotocol.com/spec
- Bookmark the official documentation
- Star the SDK repositories

This Week

- Clone the ACP SDK for your language
- Run the 14-line example and stream responses
- Read the four-layer architecture guide

This Month

- Integrate ACP into your editor or tool
- Test permission strategies in dev/stage
- Merge to production with policy gates

🔑 Key Takeaway

One flag. One agent. Ship safely with ACP.

📚 References

📖 Official Documentation

ACP Protocol Overview
Complete specification

Copilot CLI ACP Server
Integration reference

Four-Layer Architecture
Design deep dive

🛠️ Code & Examples

ACP SDKs & Specs
TypeScript, Python, Rust, Kotlin

Orchestrator Reference
Multi-agent patterns

SDP

Your Tool Just Got an AI Agent

Build with ACP. Ship safely. Scale.

Start here: github.com/agentclientprotocol/spec